

Reporte de Pruebas (POSTMAN)

Prueba: Servicio SOAP

Método: POST

URL: http://localhost:8081

Body (raw XML):

```
<soapenv:Envelope
xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/">
  <soapenv:Body>
    <mat:obtenerAlumnoPorMatricula
xmlns:mat="http://uav.mx/matriculas">
      <mat:matricula>UV001</mat:matricula>
    </mat:obtenerAlumnoPorMatricula>
  </soapenv:Body>
</soapenv:Envelope>
```

Resultado esperado (fragmento):

```
<return>{"matricula":"UV001","nombre":"Ana Pérez","programa":"Ingeniería de
Software"}</return>
```

Prueba: API REST Unificada

1) GET Alumno + Calificaciones

URL:

http://localhost:8081/api/alumnos/UV001

Respuesta esperada:

```
{
  "alumno": {
    "matricula": "UV001",
    "nombre": "Ana Pérez",
    "programa": "Ingeniería de Software"
  },
  "calificaciones": [
    {
      "id": 1,
      "materia": "Programación I",
      "calificacion": 95
    }
  ]
}
```

```
}
```

2) POST Nueva Calificación

POST:

`http://localhost:8081/api/calificaciones`

```
{
  "matricula": "UV001",
  "materia": "Bases de Datos",
  "calificacion": 90
}
```

Respuesta:

```
{
  "id": 6,
  "matricula": "UV001",
  "materia": "Bases de Datos",
  "calificacion": 90
}
```

SCRIPT POSTGRESQL

```
-- =====
-- TABLA: alumnos
-- =====

CREATE TABLE alumnos (
    matricula VARCHAR(20) PRIMARY KEY,
    nombre VARCHAR(150) NOT NULL,
    programa VARCHAR(150) NOT NULL
);

-- =====
-- TABLA: calificaciones
```

-- =====

```
CREATE TABLE calificaciones (  
    id SERIAL PRIMARY KEY,  
    matricula VARCHAR(20) NOT NULL,  
    materia VARCHAR(150) NOT NULL,  
    calificacion NUMERIC(5,2) NOT NULL,
```

```
    CONSTRAINT fk_calif_alumno
```

```
        FOREIGN KEY (matricula)
```

```
        REFERENCES alumnos (matricula)
```

```
        ON UPDATE CASCADE
```

```
        ON DELETE CASCADE
```

```
);
```

-- =====

-- DATOS DE EJEMPLO (alumnos)

-- =====

```
INSERT INTO alumnos (matricula, nombre, programa) VALUES  
( 'UV001', 'Ana Pérez', 'Ingeniería de Software'),  
( 'UV002', 'Luis Hernández', 'Ingeniería Mecatrónica'),  
( 'UV003', 'María López', 'Ciencias de la Computación');
```

-- =====

-- DATOS DE EJEMPLO (calificaciones)

-- =====

```
INSERT INTO calificaciones (matricula, materia, calificacion) VALUES  
( 'UV001', 'Programación I', 95),  
( 'UV001', 'Cálculo Diferencial', 88),  
( 'UV002', 'Física I', 78),
```

('UV003', 'Bases de Datos', 92),

('UV003', 'Álgebra Lineal', 85);

Diseño de la Arquitectura Unificada Basada en Servicios

Cliente del Sistema: Aplicación Web / Móvil

El cliente principal del sistema consiste en una **Aplicación Web y Móvil de la Universidad Veracruzana**, cuyo objetivo es permitir a los estudiantes consultar información académica relacionada con:

- sus datos personales (nombre, matrícula, programa),
- sus materias inscritas,
- y sus calificaciones.

La aplicación no debe comunicarse directamente con múltiples sistemas independientes, ya que eso generaría acoplamiento rígido, duplicación de lógica y dificultades de mantenimiento.

Por ello, en la arquitectura propuesta el cliente interactúa únicamente con un **punto de entrada unificado**: el API REST central.

API REST Unificada (Spring Boot)

La **API REST unificada** funciona como el componente central de la arquitectura SOA propuesta. Este servicio se desarrolla con Java y Spring Boot, y expone endpoints RESTful tal como:

GET /api/alumnos/{matricula}

El rol de esta API es:

1. **Orquestar la comunicación entre subsistemas heterogéneos.**
2. **Unificar formatos:** recibe datos desde un servicio SOAP (XML + JSON) y desde la BD (JPA + SQL) y los transforma en un JSON limpio y estandarizado.
3. **Proveer un único punto de acceso para los clientes.**
4. **Reducir el acoplamiento entre aplicaciones y servicios internos.**

Funciona como un **Service Facade**, encapsulando toda la complejidad de integración para entregar al cliente una respuesta simple y homogénea.

Integración con el Servicio SOAP de Matrículas (Python + Spyne)

El sistema original de matrículas está implementado en Python utilizando Spyne y expone un servicio SOAP que no puede ser modificado profundamente por ser un sistema legado. Para mantener compatibilidad y asegurar integración efectiva, el API REST unificado implementa un **cliente SOAP en Java** que:

1. Construye un sobre SOAP (XML) manualmente.
2. Lo envía al servicio Python en `http://localhost:8000`.
3. Recibe una respuesta SOAP que contiene un JSON embebido dentro del XML.
4. Extrae el JSON y lo convierte a un objeto Java (AlumnoModel).

Esta integración permite que la información del sistema antiguo sea utilizada por servicios modernos sin necesidad de reescribir el sistema original.

2.4. Integración con la Base de Datos de Calificaciones (Spring Boot + JPA)

El módulo de calificaciones se implementa como un servicio REST dentro de la misma API unificada.

Este componente utiliza **Spring Data JPA** para interactuar con la tabla:

- calificaciones

La API ejecuta consultas SQL a través de un repositorio:

`findByMatricula(matricula)`

Esto permite recuperar todas las calificaciones de un alumno de forma eficiente y segura.

La BD usada (PostgreSQL en Railway) actúa como **fuentes de datos estructurados**, compartida entre ambos servicios.

2.5. Proceso Completo de Orquestación

Cuando la App Web/Móvil solicita:

`GET /api/alumnos/UV001`

La API unificada realiza estos pasos:

1. **Llama al Servicio SOAP** → obtiene datos del alumno (matrícula, nombre, programa).
2. **Consulta la BD vía JPA** → obtiene todas las calificaciones del alumno.
3. **Integra resultados** en una sola respuesta JSON:
4. {
5. "alumno": {...},

6. "calificaciones": [...]
7. }
8. **Envía la respuesta final al cliente**, completamente unificada y estandarizada.

2.6. Ventajas de la Arquitectura Propuesta (SOA)

- **Interoperabilidad:** SOAP (XML) y REST (JSON) conviven sin conflictos.
- **Bajo acoplamiento:** El cliente solo conoce un endpoint unificado.
- **Reutilización:** La API REST puede ser usada por web, móvil y futuros servicios.
- **Evolución gradual:** Permite modernizar módulos uno por uno sin romper el sistema completo.
- **Mantenimiento simplificado:** Si se modifica el servicio SOAP o la BD, el cliente no se ve afectado.



